UNIDAD 6
PUNTEROS

Por Félix Ferreiras
y Zobiesky Ovalle

Punteros y Asignación Dinámica de Memoria

Objetivos

En esta unidad usted aprenderá a:

- Hacer uso de punteros
- Entender la relación entre punteros y arreglos así como los tipos de operaciones que pueden ser realizadas con ellos
- Utilizar punteros con funciones
- Trabajar con asignación dinámica de memoria para crear arreglos de tamaño fijo en tiempo de ejecución

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

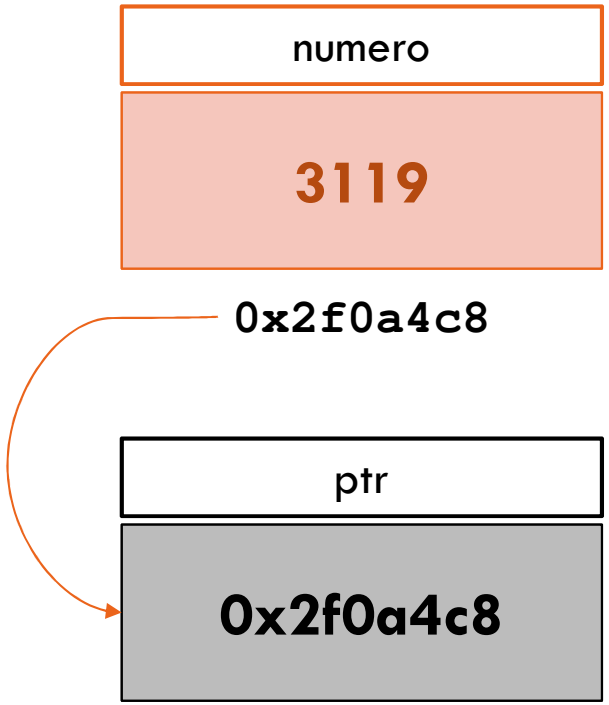
Variables tipo puntero

- &
- *

Variables tipo puntero

- Las variables tipo puntero o simplemente los punteros están diseñados para almacenar direcciones de memoria
- Los punteros nos permiten manipular de forma indirecta la data almacenada en otra variable
- Para obtener la dirección de memoria de una variable utilizamos el operador &

Variables tipo puntero




```
int numero = 3119;
cout << &numero << endl;


int * ptr;
ptr = &numero;

cout << ptr << endl;
cout << *ptr << endl;
```

Declaramos una variable tipo puntero llamada ***ptr*** la cual almacena la dirección de memoria de la variable ***numero***. Esto significa que ***ptr*** sabe donde se encuentra el valor **3119** y puede alterarlo.

Variables tipo puntero

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    int numero = 3119;
    int *ptr = &numero;

    cout << "Direccion de numero: " << &numero << endl;
    cout << "Contenido de ptr   : " << ptr << endl;

    cout << "Valor de numero    : " << *ptr << endl;
    return 0;
}
```

```
F:\cpp\pointer1.exe
Direccion de numero: 0x22fef8
Contenido de ptr   : 0x22fef8
Valor de numero    : 3119

Process returned 0 (0x0)   execution time : 0.011 s
Press any key to continue.
```

En el siguiente ejemplo podemos ver como a través del puntero podemos acceder a la variable cuya dirección de memoria está almacenada en la variable puntero.

Note que para acceder al valor al que apunta el puntero debemos utilizar el operador * como prefijo en la variable tipo puntero, de esta forma podríamos alterar también dicho valor, por ejemplo:

```
*ptr += 10;
```

Incrementaría el valor de ***numero*** en 10.

Variables tipo puntero

- Las variables punteros requieren que se especifique el tipo de dato que va a apuntar (a menos que sean punteros void)
- Un puntero válido se puede asignar a otro puntero del mismo tipo
- Un puntero es válido si al momento de usarlo, éste ya tiene una dirección de memoria almacenada

Punteros void

- En C++, void representa la ausencia de tipo, de manera que los punteros void son punteros que apuntan a un valor que no tiene tipo y por lo tanto de una longitud indeterminada
- Esto permite que los punteros void puedan apuntar a cualquier tipo de datos
- La data apuntada por ellos no puede ser directamente desreferenciada

Punteros void

- Siempre será necesario hacer *casting* a la dirección de memoria en el puntero void hacia otro tipo de puntero que apunte a un tipo de dato concreto antes de desreferenciarlo
- Uno de sus usos puede ser pasar valores genéricos a una función, una función que pueda recibir cualquier tipo de argumento

Punteros void

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    int numero = 3119;
    void *ptr = &numero;

    cout << "Direccion de numero: " << &numero << endl;
    cout << "Contenido de ptr   : " << ptr << endl;

    int *pInt = (int*) ptr;
    cout << "Valor de numero    : " << *pInt << endl;
    return 0;
}
```

```
F:\cpp\pointer1.exe
Direccion de numero: 0x22fef8
Contenido de ptr   : 0x22fef8
Valor de numero    : 3119

Process returned 0 (0x0)   execution time : 0.011 s
Press any key to continue.
```

En este ejemplo se demuestra el uso de punteros ***void***. Es necesario hacer ***casting*** para poder manipular el valor concreto a un puntero del mismo tipo de dato que el contenido en la dirección de memoria de la variable referenciada.

Puntero constante

- Un puntero constante es un puntero que no puede apuntar a otra dirección de memoria que no sea la que en su inicialización le fue asignada.
- Un puntero constante puede modificar la data contenida en la dirección de memoria a la cual el puntero apunta, siempre y cuando esta data no sea también constante.
- La sintaxis para declarar un puntero constante y conseguir que el puntero sea inalterable, se debe colocar el especificador **const** a la derecha del * Ej:

char * **const** prtChr = &letra;

Puntero constante

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      char letra = 'M';
8      char vocal = 'A';
9      char * const ptrChr = &letra;
10
11     /**
12      El puntero puede modificar el contenido
13      de la direccion de memoria que el apunta
14     */
15     *ptrChr = 'Z'; // Correcto!
16
17     /**
18      El puntero NO puede apuntar a otra direccion
19      de memoria, su valor no puede ser cambiado
20     */
21     ptrChr = &vocal; // Error!!!
22     return 0;
23 }
```

El puntero puede cambiar la data almacenada en la dirección de memoria que tiene asignada.

El puntero NO puede apuntar a otra dirección de memoria, esto es, la variable puntero no puede ser alterada o modificada, ***es una constante***.

Puntero a constante

- Un puntero a constante es aquel que apunta a una dirección de memoria cuyo contenido almacenado en ella no puede ser alterado por el puntero.
- Un puntero a constante ***no*** puede modificar la data contenida en la dirección de memoria a la cual el puntero apunta.
- La sintaxis para declarar un puntero a constante y de tal forma no pueda alterar el dato al cual a punta, se debe colocar el especificador **const** a la izquierda del * y el tipo de dato. Ej:

const char * prtChr = &letra;

Puntero a constante

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      char letra = 'M';
8      char vocal = 'A';
9      const char * ptrChr = &letra;
10
11     /**
12      El puntero NO puede modificar el contenido
13      de la direccion de memoria que el apunta
14     */
15     *ptrChr = 'Z'; // Error!!!
16
17     /**
18      El puntero puede apuntar a otra direccion
19      de memoria, su valor puede ser cambiado
20     */
21     ptrChr = &vocal; // Correcto!
22     return 0;
23 }
```

El puntero NO puede cambiar la data almacenada en la dirección de memoria que tiene asignada.

El puntero puede apuntar a otra dirección de memoria, esto es, la variable puntero puede ser alterada o modificada.

Puntero constante y puntero a constante

- Esto representa un puntero que no puede alterar el valor al cual éste apunta y tampoco puede apuntar a otra dirección de memoria.
- Es un puntero constante y también un puntero a constante.
- La sintaxis para declarar un puntero constante y a la vez sea un puntero a constante, se debe colocar el especificador **const** a la derecha * y también a la izquierda el tipo de dato. Ej:

```
const double * const pointer = &sueldo;
```

Constante de puntero y puntero a constante

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       double sueldo = 57900;
8       double sueldoAnual = sueldo * 12;
9
10      const double * const pointer = &sueldo;
11
12      /**
13          No se puede modificar el contenido
14          de la data a la cual apunta el puntero,
15          es un puntero de constante
16      **/
17      *pointer *= 1.10;
18
19      /**
20          No se puede cambiar el valor de la
21          variable puntero, es una constante de
22          puntero
23      **/
24      pointer = &sueldoAnual;
25
26      cout << *pointer << endl;
27
28      return 0;
29  }
```

El puntero NO puede cambiar la data almacenada en la dirección de memoria que tiene asignada.

El puntero NO puede apuntar a otra dirección de memoria, esto es, la variable puntero no puede ser alterada o modificada, ***es una constante***.

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

Relación entre punteros y arreglos

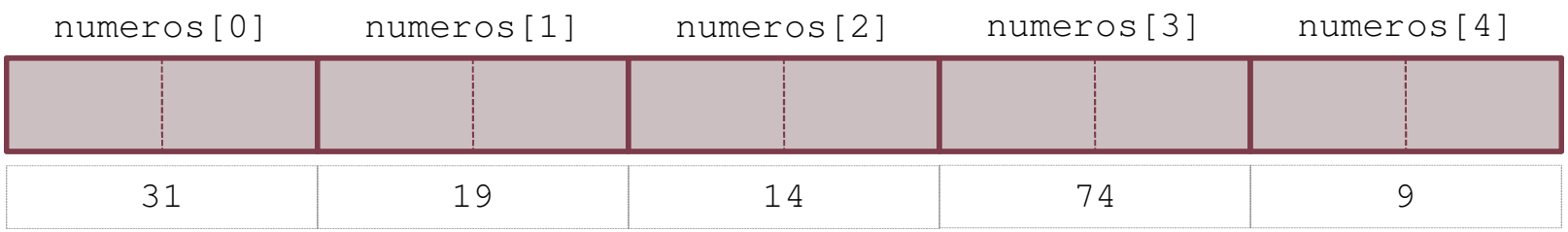
*

[ ]

Relación entre punteros y arreglos

- Los nombres de arreglo pueden ser usados como punteros
- Los punteros pueden ser usados como nombres de arreglo
- Cuando trabajamos con arreglos y utilizamos el operador de corchetes y un índice para indicar cual elemento del arreglo será accedido efectivamente estamos haciendo un movimiento de una posición de memoria a otra la cual puede ser en secuencia o aleatoria
- Los nombres de arreglo no requieren el operador & para hacer referencia a la dirección de memoria del arreglo. El nombre del arreglo contiene la dirección de memoria del primer elemento del arreglo

Relación entre punteros y arreglos




```
short numeros[] = {31, 19, 14, 74, 9};
```

En la línea anterior declaramos un arreglo tipo ***short*** (2 bytes) de cinco elementos los cuales ya fueron asignados en la definición del arreglo.

El nombre del arreglo puede ser utilizado perfectamente como un puntero, ahora bien, surge la siguiente pregunta: Si el nombre del arreglo es un puntero que contiene la dirección de memoria del primer elemento de éste **¿Cómo puedo acceder a las demás posiciones utilizando el nombre del arreglo como puntero?**

Basta con sumar el valor del índice a la variable puntero antes de utilizar asterisco:

*(numeros + 0) --- Accede al 1er elemento del arreglo
*(numeros + 1) --- Accede al 2do elemento del arreglo

Relación entre punteros y arreglos

- Es importante notar que para nuestro ejemplo

```
numeros[2] = *(numeros + 2)
```

y

```
*(numeros + 2) = *(numeros + 2 * sizeof(short))
```

- Esto significa que en realidad estamos realizando un movimiento de posición en memoria en función del tamaño en bytes del tipo de dato del arreglo, en este caso sería de 2 bytes cada movimiento, de dos en dos

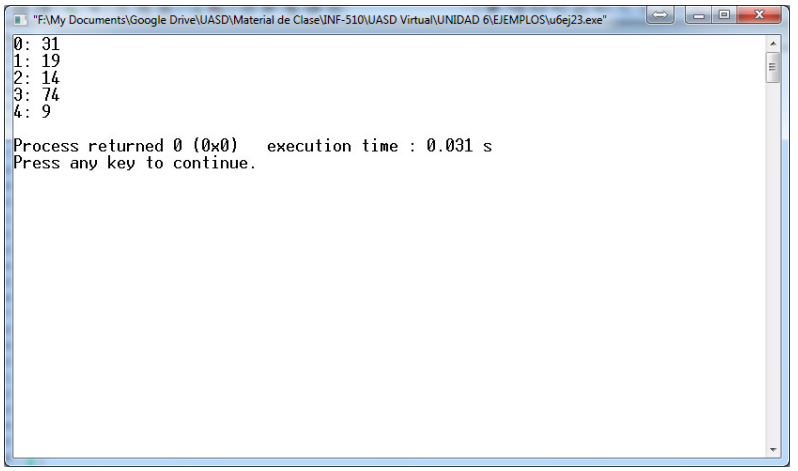
Relación entre punteros y arreglos

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      short numeros[] = {31, 19, 14, 74, 9};
8
9      /** No se debe utilizar &numeros para
10       * asignar la direccion de memoria del
11       * arreglo a numerosPtr ya que el nombre
12       * del arreglo en si mismo contiene la
13       * direccion de memoria en RAM de la primera
14       * posicion en el arreglo, el elemento cero
15       */
16     short *numerosPtr = numeros;
17
18     int indice = 0;
19     while (indice < 5)
20     {
21         cout << indice << ": "
22              << *(numerosPtr + indice)
23              << endl;
24         indice++;
25     }
26
27     return 0;
28 }
```

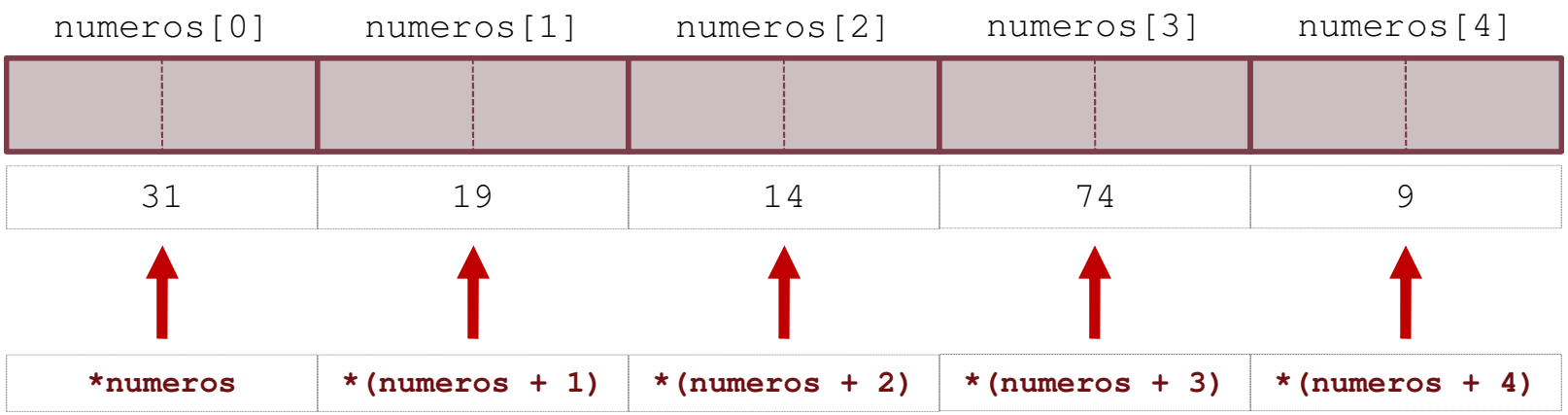
En el ejemplo se puede ver como a través del puntero podemos acceder cada uno de los elemento del arreglo, incluso podemos también modificar su contenido, por ejemplo:

```
*(numerosPtr + indice) *= 2;
```

Esto duplicaría el valor de cada elemento y lo cambiaría en el arreglo, en realidad con el puntero tenemos un acceso igual que al utilizar arreglos con operador corchetes e índice, no hay diferencias, se accede al contenido de cada dirección de memoria.



Relación entre punteros y arreglos



Se accede a los elementos del arreglo de ambas formas, más adelante veremos otras operaciones asociadas a punteros.

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

Operaciones con punteros

Aritmética de punteros

Comparaciones entre punteros

Operaciones con punteros

- Podemos realizar algunas operaciones aritméticas y de comparación con punteros
- Las operaciones aritméticas que podemos realizar son de suma o resta, lo que nos permite movernos hacia delante o atrás con relación a la posición de memoria a la que apunta el puntero
- También podemos realizar operaciones de comparación con punteros (>, <, <=, >=, ==, !=)

Operaciones con punteros

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    const unsigned int SIZE = 5;
    short numeros[SIZE] = {31, 19, 14, 74, 9};
    short * numerosPtr = numeros;

    cout << "Imprimiendo arreglo del primero al ultimo\n";
    int indice = 0;
    while (indice < SIZE)
    {
        cout << "[" << indice << "] (" << numerosPtr
             << "): " << *numerosPtr << endl;
        indice++;
        numerosPtr++;
    }

    cout << "\nImprimiendo arreglo del ultimo al primero\n";
    indice = SIZE - 1;
    numerosPtr--;
    while (indice >= 0)
    {
        cout << "[" << indice << "] (" << numerosPtr
             << "): " << *numerosPtr << endl;
        indice--;
        numerosPtr--;
    }

    return 0;
}
```

Sumando al puntero podemos movernos a las siguientes direcciones de memoria, es decir, más adelante con respecto a la dirección actual, de igual manera si restamos podemos movernos hacia a posiciones anteriores.

Luego del primer ciclo debemos restar 1 a la variable puntero ya que la operación de post-incremento dejo la variable apuntando a una dirección de memoria luego de la ultima.

```
"C:\Users\Zobiesky Ovalle\Google Drive\UASD\Material de Clase\INF-510\UASD Virtual\UNIDAD 6\EJEMPLOS\u6ej27.exe"
Imprimiendo arreglo del primero al ultimo
[0] (0x28feea): 31
[1] (0x28feec): 19
[2] (0x28feee): 14
[3] (0x28fef0): 74
[4] (0x28fef2): 9

Imprimiendo arreglo del ultimo al primero
[4] (0x28fef2): 9
[3] (0x28fef0): 74
[2] (0x28feee): 14
[1] (0x28feec): 19
[0] (0x28feea): 31

Process returned 0 (0x0)   execution time : 0.045 s
Press any key to continue.
```

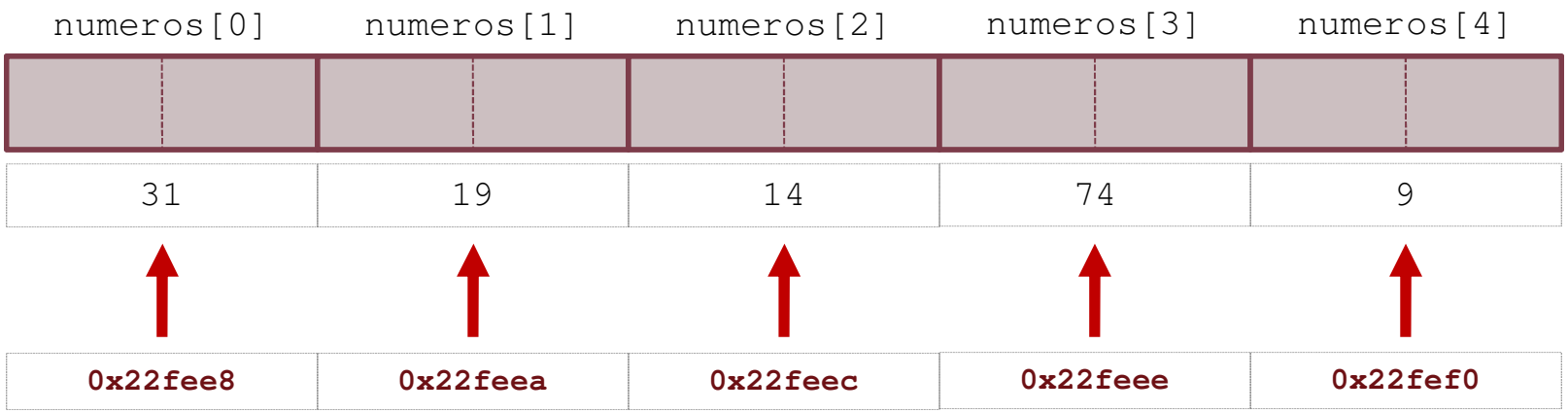
Operaciones con punteros

- De igual forma podemos realizar operaciones de relación o comparación con los punteros, veamos la siguiente tabla siendo punteroA y punteroB dos variables tipo puntero con direcciones de memoria distintas:

Operación	Resultado
`punteroA > punteroB`	El punteroA está después del punteroB
`punteroA < punteroB`	El punteroA está antes del punteroB
`punteroA >= punteroB`	El punteroA está después o en la misma posición que punteroB
`punteroA <= punteroB`	El punteroA está antes o en la misma posición que punteroB
`punteroA == punteroB`	El punteroA y el punteroB son iguales, apuntan a la misma dirección de memoria, acceden a la misma data
`punteroA != punteroB`	El punteroA y el punteroB tienen direcciones distintas

Operaciones con punteros

- En la siguiente imagen podemos ver los valores de cada dirección de memoria según el elemento o posición en el arreglo:



Operaciones con punteros

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       const unsigned int SIZE = 5;
8       short numeros[SIZE] = {31, 19, 14, 74, 9};
9       short *numerosPtr = numeros;
10
11      short *punteroA = numerosPtr + 1;
12      short *punteroB = numerosPtr + 4;
13
14      cout << "Direccion punteroA: " << punteroA << endl;
15      cout << "Direccion punteroB: " << punteroB << endl;
16      cout << endl;
17
18      if (punteroA > punteroB)
19          cout << "punteroA es MAYOR que punteroB";
20      else if (punteroA == punteroB)
21          cout << "puntero es IGUAL que punteroB";
22      else
23          cout << "punteroA es MENOR que punteroB";
24
25      cout << endl;
26
27      return 0;
28  }
```

Observe que punteroA tiene una dirección de memoria menor a la que tiene el punteroB, está 6 bytes más hacia la derecha:

0x22fee8 + 2 bytes = 0x22feeea
0x22feea + 2 bytes = 0x22feeec
0x22feec + 2 bytes = 0x22feeee

El movimiento es de dos en dos por que es un puntero tipo ***short*** y por lo tanto el tamaño manejado son 16 bits (2 bytes).

```
"F:\My Documents\Google Drive\UASD\Material de Clase\INF-510\UASD Virtual\UNIDAD 6\EJEMPLOS\u6ej25.exe"
Direccion punteroA: 0x22fee8
Direccion punteroB: 0x22feee

punteroA es MENOR que punteroB

Process returned 0 (0x0)   execution time : 0.269 s
Press any key to continue.
```

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

Asignación dinámica de memoria

Arreglos de tamaño fijo en tiempo de ejecución

Memoria física

- Es la memoria que se conecta directamente al procesador y es accesible solo por direcciones físicas; otros nombres son memoria real, memoria principal, RAM
- Es la cantidad total de memoria RAM instalada en el computador
- La memoria física debe ser organizada por el sistema operativo de una manera lógica para que el computador pueda utilizarla

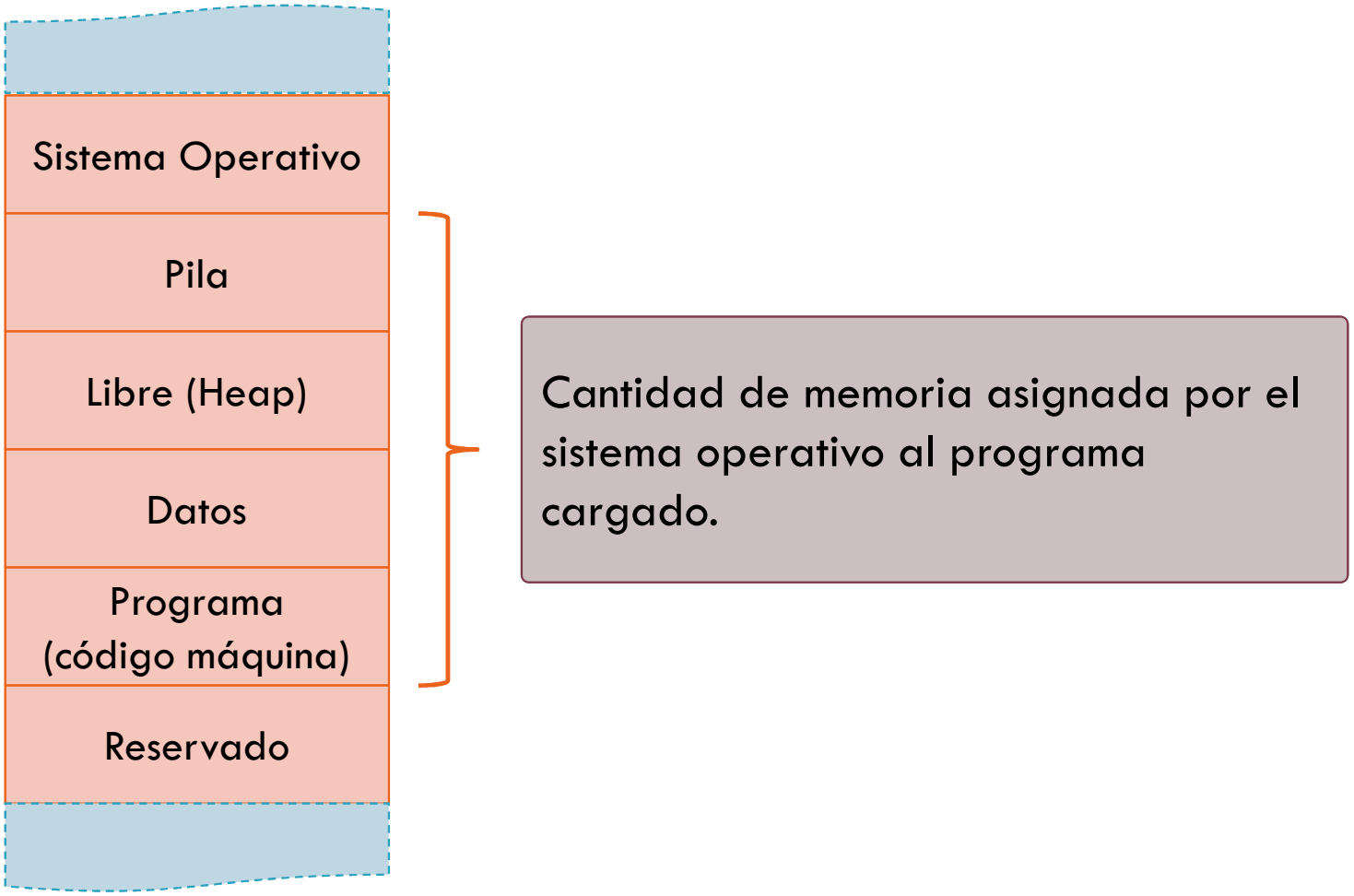
Memoria lógica

- Es la forma en que el sistema operativo organiza la memoria física
- La memoria lógica está organizada en forma de particiones con tamaños máximos y mínimos; cada partición es mapeada a la memoria física
- Típicamente el espacio de direcciones es organizado en:
  - Memoria convencional
  - Memoria superior
  - Memoria extendida

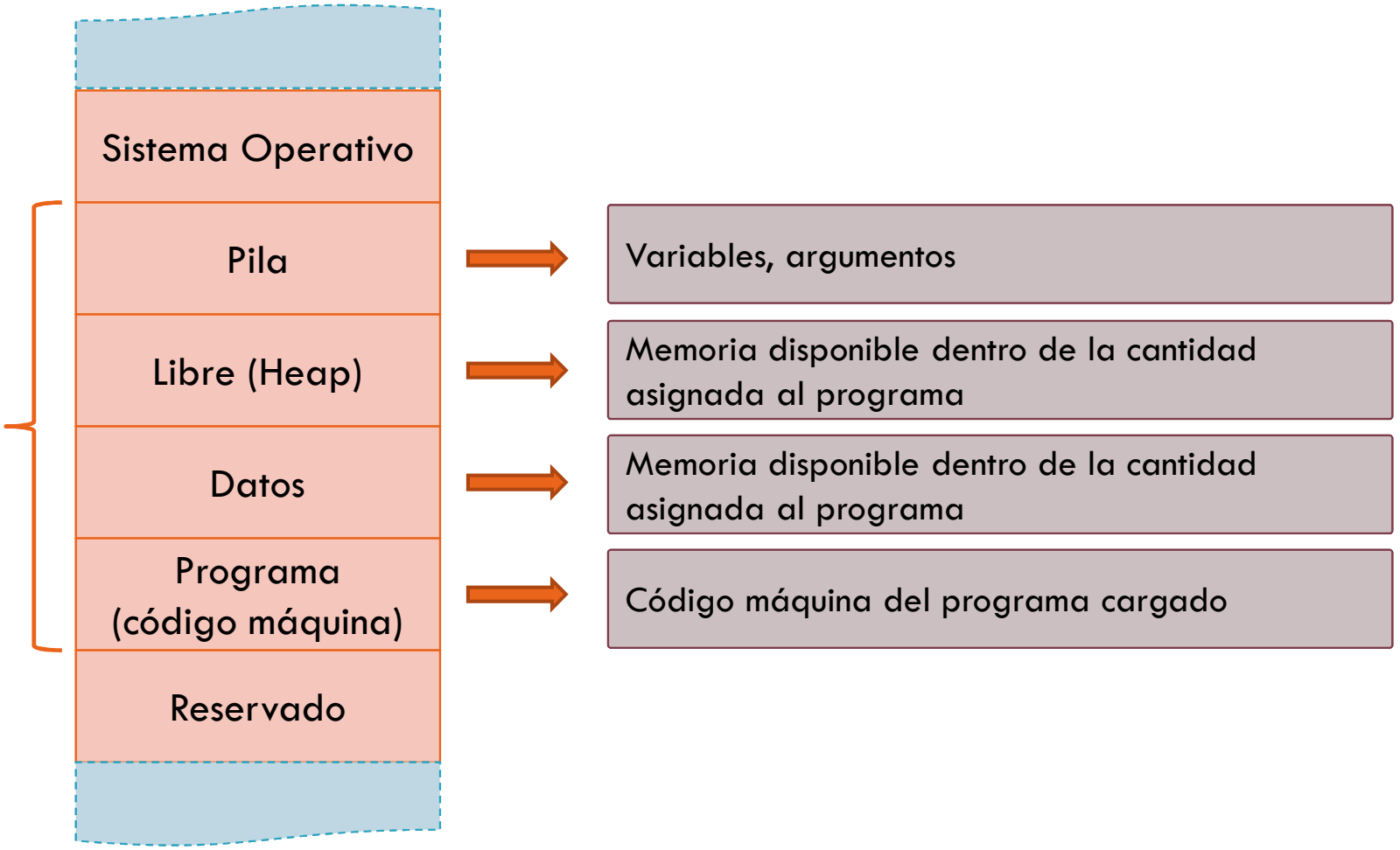
Asignación de memoria por el sistema operativo

- En la ejecución de una aplicación, el sistema operativo le asigna una cierta cantidad de memoria para:
  - Segmento de código
  - Segmento de datos
  - Segmento de pila
  - Libre (Heap)

Asignación de memoria por el sistema operativo



Asignación de memoria por el sistema operativo

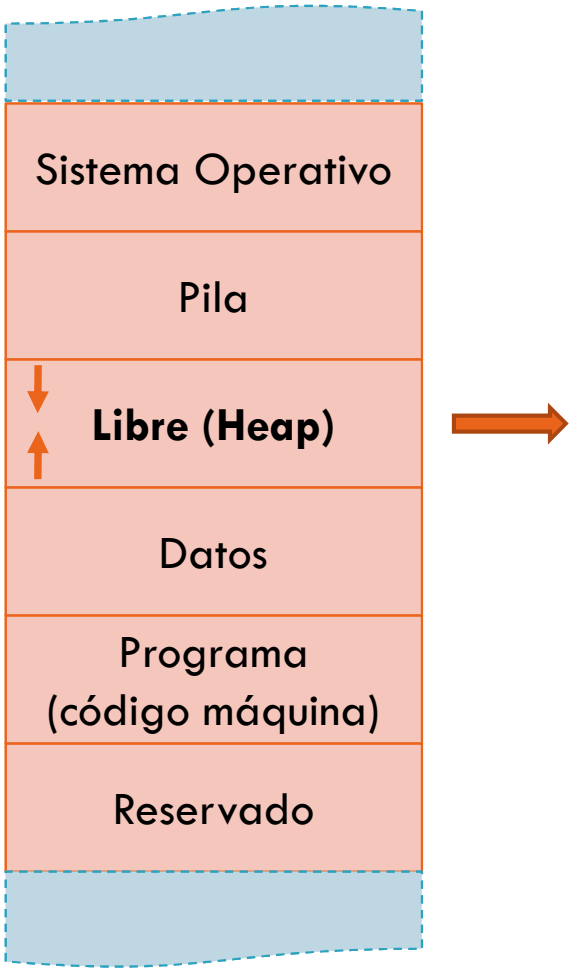


Asignación de memoria por el sistema operativo



- Es memoria lógica reservada por el sistema operativo
- Es constantemente erosionada por el sistema operativo
- Es usada solo cuando el usuario la necesita
- No almacena nombres de variables
- Es la cantidad disponible después que todo lo que está siendo ejecutado por el programa ha obtenido lo que necesita
- No tiene por que ser totalmente cargada en RAM, sino mapeada, pues parte de ella puede ser paginada (swap) al disco duro por el sistema operativo (swapping)

Asignación de memoria por el sistema operativo



- Es un espacio de direcciones cuyo tamaño lo determina el sistema operativo en función de la cantidad de RAM disponible, memoria virtual y paginación
- A medida que el programa es ejecutado, el segmento de datos crece hacia arriba y el segmento de pila crece hacia abajo
- Es la memoria lógica usada por el sistema operativo para atender los requerimientos de *memoria dinámica* (memoria que solo es determinada en tiempo de ejecución) hechos por el programa

Asignación de memoria por el sistema operativo



- Es accedida solo mediante el uso de punteros
- Es una memoria para ser usada (**new**) y liberada (**delete**) de manera responsable por el programador

Asignación dinámica de memoria

- Es cuando el programa en ejecución le requiere al sistema operativo que le asigne un bloque de memoria libre
- La memoria requerida y asignada deberá ser liberada también en tiempo de ejecución cuando ya no sea necesario su uso
- Puede ser liberada en cualquier orden, sin importar el orden en que fue asignada, esto provoca los llamados hoyos en la *heap*

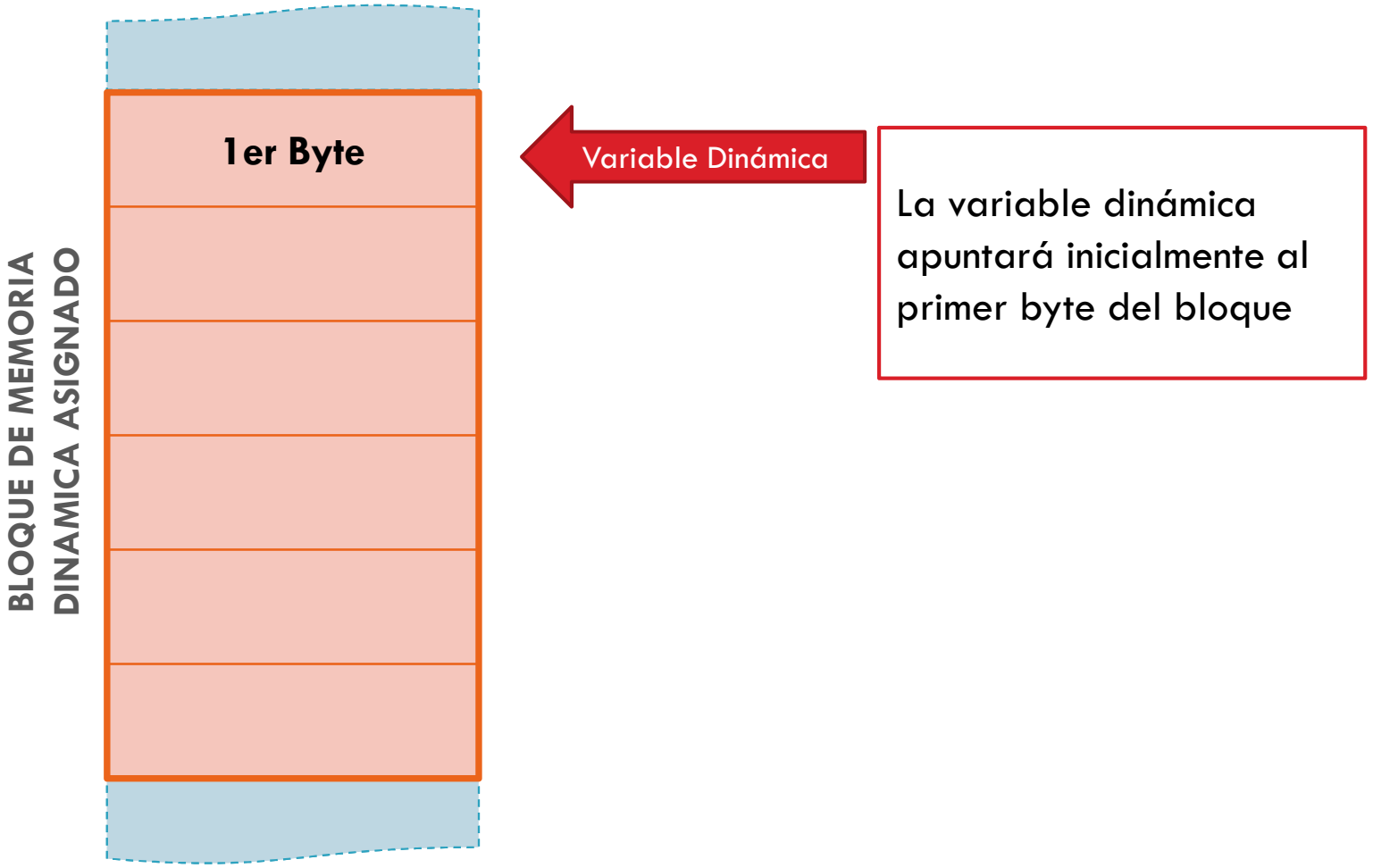
Asignación dinámica de memoria

- Un nuevo requerimiento de memoria dinámica deberá asignar un bloque completo, sin hoyos, de direcciones de memoria
- Mantenerse al tanto de la memoria asignada y liberada es complicado, un sistema operativo moderno realiza todo este trabajo

Variable dinámica

- Es una *variable puntero* cuya dirección es asignada con el operador **new** en tiempo de ejecución; la dirección asignada es tomada de la *heap*, luego *la variable apuntará a un bloque de memoria libre el cual será la llamada* memoria dinámica
- El bloque de memoria dinámica es apuntado por la variable dinámica con la que gestionará la información allí almacenada

Variable dinámica



La variable dinámica apuntará inicialmente al primer byte del bloque

Variable dinámica

- La cantidad de memoria dinámica requerida podría no estar disponible, por lo tanto, se debe siempre verificar si el sistema operativo realizó la asignación solicitada antes de usarla
- Las variables dinámicas nunca son inicializadas por el compilador, es recomendable inicializarlas manualmente
- En contraste con la variable dinámica está la variable estática, la cual tiene una dirección de memoria que es reservada en tiempo de compilación

Operadores new y delete

- Son exclusivos de C++
- Son usados para gestionar la asignación y liberación de memoria dinámica a través de una variable dinámica
- Con **new** creamos una variable dinámica, apropiándonos de recursos de memoria
- Con **delete** destruimos una variable dinámica, liberando recursos de memoria

Operadores new y delete

- Para crear una variable dinámica:

```
tipo * variable_puntero = new tipo[tamano];
```

Donde

tipo: es el tipo de dato que manejará la variable dinámica (direcciones de memoria de *tipo*), puede ser un tipo primitivo o uno definido por el programador

variable_puntero: es el nombre de la variable dinámica

tamano: es la cantidad de *tipo* que es demandada por el programa al sistema operativo

Operadores new y delete

- Para liberar el espacio utilizado por una variable dinámica:

```
delete variable_puntero;
delete [] variable_puntero;
```

  Utilizamos corchetes si la variable apunta a un arreglo asignado dinámicamente en memoria

- Solo se debe utilizar **delete** con variables previamente usadas con **new**

Asignación dinámica de memoria

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int cantidad;
8       int * arreglo;
9
10      cout << "Cantidad de elementos: ";
11      cin >> cantidad;
12      arreglo = new int[cantidad];
13
14      // inicializar cada elemento
15      for (int i = 0; i < cantidad; i++)
16          *(arreglo + i) = 0; // arreglo[i] = 0;
17
18      // imprimir el arreglo
19      for (int i = 0; i < cantidad; i++)
20          cout << "[" << i << "]: "
21               << *(arreglo + i) << endl;
22
23      delete [] arreglo;
24      arreglo = 0; // arreglo = NULL;
25
26      return 0;
27  }
```

Declaramos una variable tipo puntero

Requerimos al sistema operativo que asigne una cantidad de memoria de forma dinámica, dicha cantidad es desconocida en tiempo de compilación

Este bloque de código nos sirve para inicializar cada posición de memoria creada dinámicamente. C++ no inicializa la los espacios de memoria que son creados. Notar que puede acceder a las posiciones de memoria con notación puntero o con notación arreglo

Este ciclo se encarga de imprimir el contenido de cada posición de memoria, puede usar notación puntero o notación arreglo

Liberamos el espacio de memoria haciendo uso del operador ***delete*** y asignamos cero (puede usar ***NULL***) a la variable para remover toda referencia de memoria

Asignación dinámica de memoria

- C++ no provee un método para inicializar los arreglos en la memoria dinámica; se recomienda usar un bucle para inicializar cada elemento individualmente
- Cuando sea necesario se recomienda utilizar los arreglos inteligentes de C++, <vector>, los cuales son intrínsicamente dinámicos en su construcción y manejo

Corrupción de memoria

- Por liberación de una variable dinámica que haya sido previamente liberada

```
double *ptr = new double(19.09);
delete ptr;
delete ptr;
```



- Por liberación de una variable a la que no le fue asignada memoria

```
double *ptr;
delete ptr;
```



Corrupción de memoria

- Por asignar un valor a una variable a la que no le fue asignada memoria

```
double *ptr;
*ptr = 1409.19;
```



- Por asignar a un valor a una variable dinámica después de que ésta fue liberada

```
double *ptr = new double(19.09);
delete ptr;
*ptr = 6.75;
```



Corrupción de memoria

- Por intentar acceder a un elemento de un arreglo usando un índice fuera de rango

```
double *ptr = new double[18];
ptr[23] = 50.01;
```



Corrupción de memoria

- Para garantizar que este problema no ocurra, C++ incorpora en su librería el concepto de *punteros inteligentes* (smart pointers) en la clase auto_ptr

Contenido

- Variables tipo puntero
- Relación entre punteros y arreglos
- Operaciones con punteros
- Asignación dinámica de memoria
- Punteros y funciones

Punteros y funciones

Como argumentos

Como tipo de retorno

Punteros como argumentos

- Podemos utilizar punteros como argumentos en funciones, veamos el prototipo de una función que lo incorpore:

```
tipo funcion(tipo * variable);

tipo funcion(tipo * variable, int tamano);
```

- Si enviamos un arreglo creado dinámicamente debemos enviar también la cantidad de elementos que se utilizó en la declaración de la variable puntero que lo aloja

Punteros como argumento

```
/*************************************************************************/
/* Funcion que imprime un arreglo, lo recibe como un puntero      */
/*************************************************************************/
void imprimeArreglo(int *arr, int size)
{
    cout << "\n\nArreglo de " << "<" << size << ">" << " elementos:\n ";
    for (int contador = 0; contador < size; contador++)
    {
        cout << arr[contador] << "    ";
    }
}
```

Porción de código de un programa que contiene una función la cual recibe un puntero el cual fue dinámicamente creado con ***new***, es decir, es un arreglo de tamaño fijo en tiempo de ejecución.

Es necesario recibir la cantidad de elementos del arreglo (***new int[cantidad]***) en la función de forma que no vayamos a incurrir en corrupción de memoria por intentar acceder a un elemento fuera de rango.

Punteros como valor de retorno

- Podemos utilizar punteros como valor de retorno en funciones, veamos el prototipo de una función que lo utilice:

```
tipo * funcion(argumentos);
```

Punteros como valor de retorno

```
/*********************************************************************/
/* Funcion que crea un arreglo de un tamano definido por el          */
/* argumento y retorna un puntero que hace referencia a la           */
/* direccion en memoria donde se encuentra                           */
/*********************************************************************/
int *creaArreglo(int size)
{
    int *arr;  // Arreglo para almacenar los numeros

    // Retorna NULL si size es cero o negativo
    if (size <= 0)
        return NULL;

    // Asigna dinamicamente el arreglo
    arr = new int[size];
    if (arr != NULL)
    {
        // Coloca la semilla para la generacion aleatoria
        // de los numeros
        srand(time(0));
        // Poblar el arreglo con numeros aleatorios
        for (int contador = 0; contador < size; contador++)
            arr[contador] = 1 + rand() % size;
    }

    // retorna el puntero
    return arr;
}
```

Esta función se encarga de crear un arreglo utilizando asignación dinámica de memoria, con el tamaño especificado en el argumento ***size***.

La función valida si la cantidad recibida como argumento es menor o igual a cero para que la solicitud de asignación de memoria no genere error.

Luego de hacer el requerimiento validamos si éste fue exitoso y de ser así procedemos a inicializar el arreglo con valores aleatorios.

Finalmente retornamos el puntero, esto es, retornamos la dirección de memoria del primer elemento.

Resumen

- En esta unidad hemos aprendido:
  - Que los punteros son un tipo de variables que pueden almacenar direcciones de memoria
  - A través de punteros podemos alterar de forma indirecta el valor contenido en otra variable
  - Los nombres de arreglo son punteros y los punteros pueden ser utilizados como arreglos
  - Podemos utilizar aritmética de punternos para movernos a través de las distintas posiciones en un arreglo
  - Podemos realizar operaciones de comparación con punteros, los que nos permite determinar si una posición de memoria en un puntero está antes o después que otra

Resumen

- En esta unidad hemos aprendido:
  - Mediante la asignación dinámica de memoria podemos crear arreglos de tamaño fijo en tiempo de ejecución
  - Es importante liberar el espacio utilizado previamente requerido mediante el operador ***new***
  - Los punteros pueden ser utilizados como argumentos o tipo de retorno en funciones

Bibliografía

- Addison Wesley, Tony Gaddis. (2009). Starting Out with C++ Brief: From Control Structures through Objects. San Francisco, California. 6th Edition.
- Aprende a programar en C++. (2010). Página dedicada a la enseñanza de C++ con libro de libre descarga, recuperado el 3 Junio 2013 de: http://c.conclase.net/curso/index.php
- Página dedicada al lenguaje de programación C++ (2000 – 2014). Tutoriales y recursos de libre descarga, recuperado Enero 2014 en: http://www.cplusplus.com All rights reserved - v3.1
- Paul Deitel, Harvey Deitel (2011). C++ How to Program, New York, 8th Edition, Prentice Hall.



Unidad 6: Punteros

Por Félix Ferreiras y Zobiesky Ovalle